

SGT UNIVERSITY

School of Engineering and Technology

Department of CSE

DATABASE MANAGEMENT SYSTEMS LABORATORY

Experiment 4

SELECT Queries — DISTINCT, WHERE, BETWEEN, IN, ORDER BY

Field	Details
Student	Harshit Khemani
Roll No.	241302081
Programme	B.Tech CSE (AI/ML)
Section	Section - C
Lab Date	09 September 2026
Dataset	employee — 10 sample rows × 8 attributes
Methods	SELECT, DISTINCT, WHERE, BETWEEN, IN, ORDER BY

1. Aim

To answer the lab questions below: create an employee table with the given attributes, insert 10 rows, and run the listed SELECT queries (DISTINCT, WHERE, BETWEEN, IN, ORDER BY).

2. Theory

2.1 SELECT

SELECT reads rows from a relation. SELECT * returns every column; naming columns returns only those attributes. A query without a WHERE clause returns every row that currently exists in the table.

2.2 DISTINCT

SELECT DISTINCT removes duplicate values from the result. Question 3 on the lab sheet asks for a “distant” department number; that is DISTINCT. SELECT department_no lists a department once per employee (question 4); SELECT DISTINCT department_no lists each department number only once.

2.3 WHERE

The WHERE clause keeps rows that satisfy a condition. Comparison operators such as > and <> filter numbers and strings. OR keeps a row if either predicate is true (city is Delhi or Mumbai). City values in this lab are stored in title case and matched with the same casing so the filters work in SQLite without extra case-conversion functions.

2.4 BETWEEN

BETWEEN low AND high is an inclusive range: the endpoints are part of the result. NOT BETWEEN is the complement — salaries strictly below 30000 or strictly above 50000. It is equivalent to salary < 30000 OR salary > 50000 when the range is 30000 to 50000.

2.5 IN

IN (...) tests membership in a list. city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore') keeps those four cities. NOT IN keeps every city that is not in the list — here Bangalore, Hyderabad, and Pune remain when Mumbai, Delhi, and Chennai are excluded.

2.6 ORDER BY

ORDER BY sorts the result. ASC (the default) lists names A→Z; DESC lists them Z→A. Sorting does not change stored rows; it only changes the order of the result set.

3. Schema

One table, employee, holds eight attributes. sr_no is the employee number (primary key). manager stores the manager's name, or NULL for the top manager.

Attribute	Type	Constraint	Role
sr_no	INTEGER	PRIMARY KEY	Employee number
employee_name	TEXT	NOT NULL	Employee name
job	TEXT	NOT NULL	Job title
manager	TEXT	NULL allowed	Manager name
hire_date	DATE	NOT NULL	Date of joining
salary	NUMERIC(10,2)	NOT NULL, CHECK > 0	Monthly salary
department_no	INTEGER	NOT NULL	Department number
city	TEXT	NOT NULL	Work city

4. Questions

The experiment questions are reproduced as written on the lab sheet (two items numbered 1). Question 3 writes “distant”; the SQL keyword is DISTINCT. Employee number is stored as sr_no.

Q. No.	Question (as given)
1	create a table employee with the following attributes: sr no, employee name, job, manager, hire date, salary, department no, city
1	insert 10 rows in the above mentioned table
2	select all the details from the employee table
3	select only distant department number from employee table
4	select all department numbers from employee table
5	select employee name, salary where salary is >30000 rs from employee table
6	select employee name, hire date where city is not delhi from employee table
7	select employee name, hire date where city is either delhi or mumbai from employee table
8	select employee number, employee name where salary is between 30000 rs and 50000 rs
9	select employee number, employee name where salary is not between 30000 and 50000 from employee table
10	select all the details where city is among the following: delhi, mumbai, chennai, Bangalore
11	select all the details where city is not mumbai, delhi or chennai from employee table
12	select all the details from the employee table and order by employee names in descending order.
13	display the list of employees in ascending order from employee table.

5. Procedure

Drop employee if it already exists so the script can be re-run. Then answer each lab question in order. City values are stored in title case (Delhi, Mumbai, ...) and matched with that same casing. The SQL for each question:

Q. No.	SQL solution
1	<pre>CREATE TABLE employee (sr_no INTEGER PRIMARY KEY, employee_name TEXT NOT NULL, job TEXT NOT NULL, manager TEXT, hire_date DATE NOT NULL, salary NUMERIC(10, 2) NOT NULL CHECK (salary > 0), department_no INTEGER NOT NULL, city TEXT NOT NULL);</pre>
1	<pre>INSERT INTO employee (sr_no, employee_name, job, manager, hire_date, salary, department_no, city) VALUES (1, 'Rajesh Kumar', 'General Manager', NULL, '2018-03-12', 85000.00, 10, 'Delhi'), (2, 'Ananya Sharma', 'Software Engineer', 'Rajesh Kumar', '2021-07-01', 45000.00, 20, 'Bangalore'), (3, 'Rohan Mehta', 'Accountant', 'Rajesh Kumar', '2020-01-15', 28000.00, 30, 'Mumbai'), (4, 'Priya Nair', 'HR Executive', 'Rajesh Kumar', '2022-04-20', 32000.00, 40, 'Chennai'), (5, 'Vikram Singh', 'Sales Manager', 'Rajesh Kumar', '2019-11-08', 52000.00, 50, 'Hyderabad'), (6, 'Fatima Khan', 'Software Engineer', 'Ananya Sharma', '2023-02-14', 38000.00, 20, 'Bangalore'), (7, 'Arjun Patel', 'Clerk', 'Rohan Mehta', '2024-06-01', 18000.00, 30, 'Pune'), (8, 'Neha Gupta', 'Marketing Analyst', 'Rajesh Kumar', '2021-09-10', 35000.00, 60, 'Delhi'), (9, 'Karan Iyer', 'Database Admin', 'Ananya Sharma', '2020-12-05', 48000.00, 20, 'Mumbai'), (10, 'Meera Joshi', 'Receptionist', 'Priya Nair', '2023-08-22', 22000.00, 40, 'Chennai');</pre>
2	<pre>SELECT * FROM employee;</pre>
3	<pre>SELECT DISTINCT department_no FROM employee;</pre>
4	<pre>SELECT department_no FROM employee;</pre>
5	<pre>SELECT employee_name, salary FROM employee WHERE salary > 30000;</pre>
6	<pre>SELECT employee_name, hire_date FROM employee WHERE city <> 'Delhi';</pre>
7	<pre>SELECT employee_name, hire_date FROM employee WHERE city = 'Delhi' OR city = 'Mumbai';</pre>
8	<pre>SELECT sr_no, employee_name FROM employee WHERE salary BETWEEN 30000 AND 50000;</pre>
9	<pre>SELECT sr_no, employee_name FROM employee WHERE salary NOT BETWEEN 30000 AND 50000;</pre>
10	<pre>SELECT * FROM employee WHERE city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore');</pre>
11	<pre>SELECT * FROM employee WHERE city NOT IN ('Mumbai', 'Delhi', 'Chennai');</pre>
12	<pre>SELECT * FROM employee ORDER BY employee_name DESC;</pre>
13	<pre>SELECT * FROM employee ORDER BY employee_name ASC;</pre>

6. Source Code

SQLite script used in the lab compiler (09-09-2026/employee.sql). For the class SQL Server, run employee.sqlserver.sql.

```
-- =====
-- DBMS Lab · 09-09-2026
-- Employee table: CREATE, seed 10 rows, then SELECT queries
-- DISTINCT, WHERE, BETWEEN, IN, ORDER BY
-- SQLite (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run employee.sqlserver.sql instead.
--
```

```

-- Lab questions (wording as given; "distant" = DISTINCT)
-- 1. create a table employee with the following attributes:
--    sr no, employee name, job, manager, hire date, salary,
--    department no, city
-- 1. insert 10 rows in the above mentioned table
-- 2. select all the details from the employee table
-- 3. select only distant department number from employee table
-- 4. select all department numbers from employee table
-- 5. select employee name, salary where salary is >30000 rs
--    from employee table
-- 6. select employee name, hire date where city is not delhi
--    from employee table
-- 7. select employee name, hire date where city is either
--    delhi or mumbai from employee table
-- 8. select employee number, employee name where salary is
--    between 30000 rs and 50000 rs
-- 9. select employee number, employee name where salary is
--    not between 30000 and 50000 from employee table
-- 10. select all the details where city is among the following:
--    delhi, mumbai, chennai, Bangalore
-- 11. select all the details where city is not mumbai, delhi
--    or chennai from employee table
-- 12. select all the details from the employee table and
--    order by employee names in descending order.
-- 13. display the list of employees in ascending order from
--    employee table.
-- =====

DROP TABLE IF EXISTS employee;

-- -----
-- 1. create a table employee with the following attributes:
--    sr no, employee name, job, manager, hire date, salary,
--    department no, city
-- -----
CREATE TABLE employee (
    sr_no          INTEGER PRIMARY KEY,
    employee_name  TEXT NOT NULL,
    job            TEXT NOT NULL,
    manager        TEXT,
    hire_date      DATE NOT NULL,
    salary         NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
    department_no  INTEGER NOT NULL,
    city          TEXT NOT NULL
);

-- -----
-- 1. insert 10 rows in the above mentioned table
--    • salaries below, inside, and above 30000-50000
--    • cities: Delhi, Mumbai, Chennai, Bangalore, Hyderabad, Pune
--    • mixed department numbers (some duplicates)
--    • Rajesh Kumar is the top manager (manager IS NULL)
-- -----
INSERT INTO employee
(sr_no, employee_name, job, manager, hire_date, salary, department_no, city)
VALUES
(1, 'Rajesh Kumar', 'General Manager', NULL, '2018-03-12', 85000.00, 10, 'Delhi'),
(2, 'Ananya Sharma', 'Software Engineer', 'Rajesh Kumar', '2021-07-01', 45000.00, 20, 'Bangalore'),
(3, 'Rohan Mehta', 'Accountant', 'Rajesh Kumar', '2020-01-15', 28000.00, 30, 'Mumbai'),
(4, 'Priya Nair', 'HR Executive', 'Rajesh Kumar', '2022-04-20', 32000.00, 40, 'Chennai'),
(5, 'Vikram Singh', 'Sales Manager', 'Rajesh Kumar', '2019-11-08', 52000.00, 50, 'Hyderabad'),
(6, 'Fatima Khan', 'Software Engineer', 'Ananya Sharma', '2023-02-14', 38000.00, 20, 'Bangalore'),
(7, 'Arjun Patel', 'Clerk', 'Rohan Mehta', '2024-06-01', 18000.00, 30, 'Pune'),
(8, 'Neha Gupta', 'Marketing Analyst', 'Rajesh Kumar', '2021-09-10', 35000.00, 60, 'Delhi'),
(9, 'Karan Iyer', 'Database Admin', 'Ananya Sharma', '2020-12-05', 48000.00, 20, 'Mumbai'),
(10, 'Meera Joshi', 'Receptionist', 'Priya Nair', '2023-08-22', 22000.00, 40, 'Chennai');

-- -----
-- 2. select all the details from the employee table
-- -----
SELECT * FROM employee;

-- -----
-- 3. select only distant department number from employee table
--    ("distant" on the lab sheet = DISTINCT)
-- -----
-- Distant (DISTINCT) department number
SELECT DISTINCT department_no FROM employee;

-- -----
-- 4. select all department numbers from employee table
-- -----
-- All department numbers
SELECT department_no FROM employee;

-- -----
-- 5. select employee name, salary where salary is >30000 rs
--    from employee table
-- -----

```

```

-- Salary > 30000 rs
SELECT employee_name, salary
FROM employee
WHERE salary > 30000;

-----
-- 6. select employee name, hire date where city is not delhi
--    from employee table
-----
-- City is not delhi
SELECT employee_name, hire_date
FROM employee
WHERE city <> 'Delhi';

-----
-- 7. select employee name, hire date where city is either
--    delhi or mumbai from employee table
-----
-- City is either delhi or mumbai
SELECT employee_name, hire_date
FROM employee
WHERE city = 'Delhi' OR city = 'Mumbai';

-----
-- 8. select employee number, employee name where salary is
--    between 30000 rs and 50000 rs
-----
-- Salary between 30000 rs and 50000 rs
SELECT sr_no, employee_name
FROM employee
WHERE salary BETWEEN 30000 AND 50000;

-----
-- 9. select employee number, employee name where salary is
--    not between 30000 and 50000 from employee table
-----
-- Salary not between 30000 and 50000
SELECT sr_no, employee_name
FROM employee
WHERE salary NOT BETWEEN 30000 AND 50000;

-----
-- 10. select all the details where city is among the following:
--      delhi, mumbai, chennai, Bangalore
-----
-- City among delhi, mumbai, chennai, Bangalore
SELECT *
FROM employee
WHERE city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore');

-----
-- 11. select all the details where city is not mumbai, delhi
--      or chennai from employee table
-----
-- City is not mumbai, delhi or chennai
SELECT *
FROM employee
WHERE city NOT IN ('Mumbai', 'Delhi', 'Chennai');

-----
-- 12. select all the details from the employee table and
--      order by employee names in descending order.
-----
-- Order by employee names descending
SELECT *
FROM employee
ORDER BY employee_name DESC;

-----
-- 13. display the list of employees in ascending order from
--      employee table.
-----
-- Display employees in ascending order
SELECT *
FROM employee
ORDER BY employee_name ASC;

```

7. Output

The script was executed in SQLite in memory. CREATE and INSERT print a status line; each SELECT prints its result set. NULL in the manager column is shown as an empty cell.

```

SQL> DROP TABLE IF EXISTS employee;
-- OK

SQL> CREATE TABLE employee (
    sr_no          INTEGER PRIMARY KEY,
    employee_name  TEXT NOT NULL,

```

```

        job            TEXT NOT NULL,
        manager        TEXT,
        hire_date       DATE NOT NULL,
        salary          NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
        department_no   INTEGER NOT NULL,
        city            TEXT NOT NULL
    );
-- OK

SQL> INSERT INTO employee
(sr_no, employee_name, job, manager, hire_date, salary, department_no, city)
VALUES
(1, 'Rajesh Kumar', 'General Manager', NULL, '2018-03-12', 85000.00,
10, 'Delhi'),
(2, 'Ananya Sharma', 'Software Engineer', 'Rajesh Kumar', '2021-07-01', 45000.00,
20, 'Bangalore'),
(3, 'Rohan Mehta', 'Accountant', 'Rajesh Kumar', '2020-01-15', 28000.00,
30, 'Mumbai'),
(4, 'Priya Nair', 'HR Executive', 'Rajesh Kumar', '2022-04-20', 32000.00,
40, 'Chennai'),
(5, 'Vikram Singh', 'Sales Manager', 'Rajesh Kumar', '2019-11-08', 52000.00,
50, 'Hyderabad'),
(6, 'Fatima Khan', 'Software Engineer', 'Ananya Sharma', '2023-02-14', 38000.00,
20, 'Bangalore'),
(7, 'Arjun Patel', 'Clerk', 'Rohan Mehta', '2024-06-01', 18000.00,
30, 'Pune'),
(8, 'Neha Gupta', 'Marketing Analyst', 'Rajesh Kumar', '2021-09-10', 35000.00,
60, 'Delhi'),
(9, 'Karan Iyer', 'Database Admin', 'Ananya Sharma', '2020-12-05', 48000.00,
20, 'Mumbai'),
(10, 'Meera Joshi', 'Receptionist', 'Priya Nair', '2023-08-22', 22000.00,
40, 'Chennai');
-- 10 row(s) inserted

SQL> SELECT * FROM employee;
sr_no  employee_name  job                manager      hire_date  salary  department_no  city
-----
1      Rajesh Kumar    General Manager
2      Ananya Sharma   Software Engineer  Rajesh Kumar 2021-07-01  45000   20             Bangalore
3      Rohan Mehta     Accountant         Rajesh Kumar 2020-01-15  28000   30             Mumbai
4      Priya Nair      HR Executive       Rajesh Kumar 2022-04-20  32000   40             Chennai
5      Vikram Singh    Sales Manager      Rajesh Kumar 2019-11-08  52000   50             Hyderabad
6      Fatima Khan     Software Engineer  Ananya Sharma 2023-02-14  38000   20             Bangalore
7      Arjun Patel     Clerk              Rohan Mehta  2024-06-01  18000   30             Pune
8      Neha Gupta      Marketing Analyst  Rajesh Kumar 2021-09-10  35000   60             Delhi
9      Karan Iyer      Database Admin     Ananya Sharma 2020-12-05  48000   20             Mumbai
10     Meera Joshi     Receptionist       Priya Nair   2023-08-22  22000   40             Chennai

SQL> SELECT DISTINCT department_no FROM employee;
department_no
-----
10
20
30
40
50
60

SQL> SELECT department_no FROM employee;
department_no
-----
10
20
30
40
50
20
30
60
20
40

SQL> SELECT employee_name, salary
FROM employee
WHERE salary > 30000;
employee_name  salary
-----
Rajesh Kumar   85000
Ananya Sharma  45000
Priya Nair     32000
Vikram Singh   52000
Fatima Khan    38000
Neha Gupta     35000
Karan Iyer     48000

SQL> SELECT employee_name, hire_date
FROM employee
WHERE city <> 'Delhi';
employee_name  hire_date
-----
Ananya Sharma  2021-07-01
Rohan Mehta    2020-01-15
Priya Nair     2022-04-20
Vikram Singh   2019-11-08
Fatima Khan    2023-02-14
Arjun Patel    2024-06-01
Karan Iyer     2020-12-05
Meera Joshi    2023-08-22

SQL> SELECT employee_name, hire_date
FROM employee
WHERE city = 'Delhi' OR city = 'Mumbai';

```

```

employee_name  hire_date
-----
Rajesh Kumar   2018-03-12
Rohan Mehta    2020-01-15
Neha Gupta     2021-09-10
Karan Iyer     2020-12-05

```

```

SQL> SELECT sr_no, employee_name
      FROM employee
      WHERE salary BETWEEN 30000 AND 50000;

```

```

sr_no  employee_name
-----
2      Ananya Sharma
4      Priya Nair
6      Fatima Khan
8      Neha Gupta
9      Karan Iyer

```

```

SQL> SELECT sr_no, employee_name
      FROM employee
      WHERE salary NOT BETWEEN 30000 AND 50000;

```

```

sr_no  employee_name
-----
1      Rajesh Kumar
3      Rohan Mehta
5      Vikram Singh
7      Arjun Patel
10     Meera Joshi

```

```

SQL> SELECT *
      FROM employee
      WHERE city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore');

```

sr_no	employee_name	job	manager	hire_date	salary	department_no	city
1	Rajesh Kumar	General Manager		2018-03-12	85000	10	Delhi
2	Ananya Sharma	Software Engineer	Rajesh Kumar	2021-07-01	45000	20	Bangalore
3	Rohan Mehta	Accountant	Rajesh Kumar	2020-01-15	28000	30	Mumbai
4	Priya Nair	HR Executive	Rajesh Kumar	2022-04-20	32000	40	Chennai
6	Fatima Khan	Software Engineer	Ananya Sharma	2023-02-14	38000	20	Bangalore
8	Neha Gupta	Marketing Analyst	Rajesh Kumar	2021-09-10	35000	60	Delhi
9	Karan Iyer	Database Admin	Ananya Sharma	2020-12-05	48000	20	Mumbai
10	Meera Joshi	Receptionist	Priya Nair	2023-08-22	22000	40	Chennai

```

SQL> SELECT *
      FROM employee
      WHERE city NOT IN ('Mumbai', 'Delhi', 'Chennai');

```

sr_no	employee_name	job	manager	hire_date	salary	department_no	city
2	Ananya Sharma	Software Engineer	Rajesh Kumar	2021-07-01	45000	20	Bangalore
5	Vikram Singh	Sales Manager	Rajesh Kumar	2019-11-08	52000	50	Hyderabad
6	Fatima Khan	Software Engineer	Ananya Sharma	2023-02-14	38000	20	Bangalore
7	Arjun Patel	Clerk	Rohan Mehta	2024-06-01	18000	30	Pune

```

SQL> SELECT *
      FROM employee
      ORDER BY employee_name DESC;

```

sr_no	employee_name	job	manager	hire_date	salary	department_no	city
5	Vikram Singh	Sales Manager	Rajesh Kumar	2019-11-08	52000	50	Hyderabad
3	Rohan Mehta	Accountant	Rajesh Kumar	2020-01-15	28000	30	Mumbai
1	Rajesh Kumar	General Manager		2018-03-12	85000	10	Delhi
4	Priya Nair	HR Executive	Rajesh Kumar	2022-04-20	32000	40	Chennai
8	Neha Gupta	Marketing Analyst	Rajesh Kumar	2021-09-10	35000	60	Delhi
10	Meera Joshi	Receptionist	Priya Nair	2023-08-22	22000	40	Chennai
9	Karan Iyer	Database Admin	Ananya Sharma	2020-12-05	48000	20	Mumbai
6	Fatima Khan	Software Engineer	Ananya Sharma	2023-02-14	38000	20	Bangalore
7	Arjun Patel	Clerk	Rohan Mehta	2024-06-01	18000	30	Pune
2	Ananya Sharma	Software Engineer	Rajesh Kumar	2021-07-01	45000	20	Bangalore

```

SQL> SELECT *
      FROM employee
      ORDER BY employee_name ASC;

```

sr_no	employee_name	job	manager	hire_date	salary	department_no	city
2	Ananya Sharma	Software Engineer	Rajesh Kumar	2021-07-01	45000	20	Bangalore
7	Arjun Patel	Clerk	Rohan Mehta	2024-06-01	18000	30	Pune
6	Fatima Khan	Software Engineer	Ananya Sharma	2023-02-14	38000	20	Bangalore
9	Karan Iyer	Database Admin	Ananya Sharma	2020-12-05	48000	20	Mumbai
10	Meera Joshi	Receptionist	Priya Nair	2023-08-22	22000	40	Chennai
8	Neha Gupta	Marketing Analyst	Rajesh Kumar	2021-09-10	35000	60	Delhi
4	Priya Nair	HR Executive	Rajesh Kumar	2022-04-20	32000	40	Chennai
1	Rajesh Kumar	General Manager		2018-03-12	85000	10	Delhi
3	Rohan Mehta	Accountant	Rajesh Kumar	2020-01-15	28000	30	Mumbai
5	Vikram Singh	Sales Manager	Rajesh Kumar	2019-11-08	52000	50	Hyderabad

8. Results

Ten employees load successfully. Each lab question returns a non-empty result on this seed data:

Q. No.	Rows	What the result shows
1 (create)	—	Table employee created with 8 attributes
1 (insert)	10	10 rows inserted
2	10	All details from the employee table
3	6	Distant/DISTINCT department numbers: 10, 20, 30, 40, 50, 60
4	10	All department numbers (dept 20 appears three times)
5	7	employee name, salary where salary is >30000 rs
6	8	employee name, hire date where city is not delhi
7	4	employee name, hire date where city is delhi or mumbai
8	5	employee number, name where salary is between 30000 rs and 50000 rs
9	5	employee number, name where salary is not between 30000 and 50000
10	8	all details where city is delhi, mumbai, chennai, Bangalore
11	4	all details where city is not mumbai, delhi or chennai
12	10	all details ordered by employee names descending
13	10	list of employees in ascending order

Question 3 (DISTINCT) collapses ten department values to six unique numbers; question 4 lists all ten. Questions 8 and 9 partition salaries with no overlap ($5 + 5 = 10$). Question 11 keeps Bangalore, Hyderabad, and Pune — cities that are not mumbai, delhi, or chennai.

9. Conclusion

A single SELECT can project columns, remove duplicates, filter with comparison and set predicates, and sort the result. DISTINCT answers “which unique values exist?” WHERE answers “which rows match this condition?” BETWEEN and IN express ranges and lists clearly. ORDER BY changes presentation only. Together these clauses are the core of everyday retrieval in SQL.